Solution: Reverse a Queue

Problem Statement

Given a `queue`, return it after reversing its elements.

Examples:

1. **Input**: Queue = [1, 2, 3, 4, 5]
   Output: [5, 4, 3, 2, 1]
   Explanation: The input queue elements are reversed.
2. **Input**: Queue = [10, 20, 30, 40, 50]
   Output: [50, 40, 30, 20, 10]
   Explanation: The input queue elements are reversed.
3. **Input**: Queue = ["cat", "dog", "fish", "bird"]
   Output: ["bird", "fish", "dog", "cat"]
   Explanation: The input queue elements are reversed.

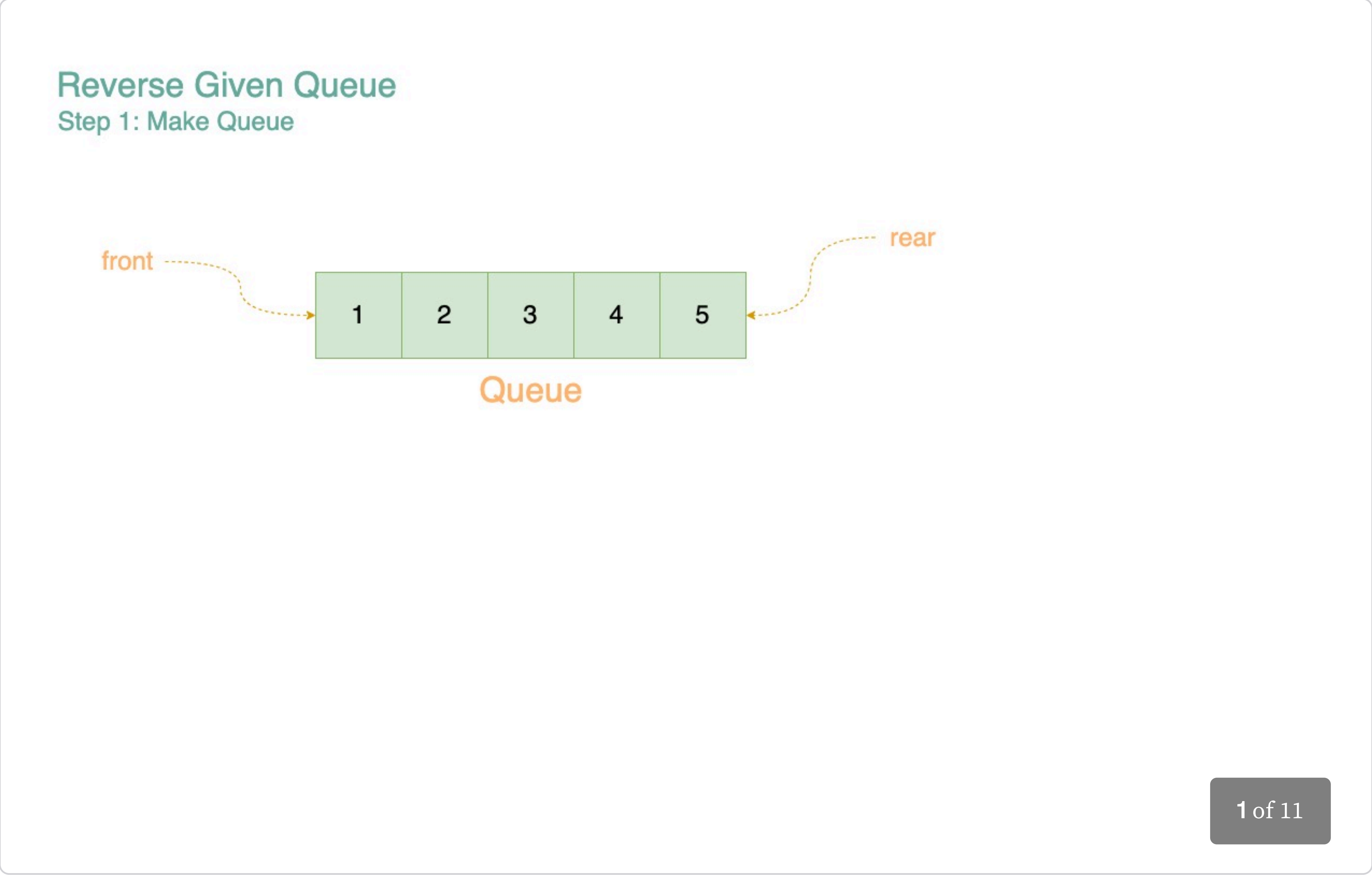
Solution

The general approach to reverse a queue involves the use of a stack. As the stack follows the Last-In-First-Out (LIFO) principle, and the queue follows the First-In-First-Out (FIFO) principle, by pushing all elements of a queue into a stack and then back into the queue, we can effectively reverse the queue.

Algorithm Walkthrough

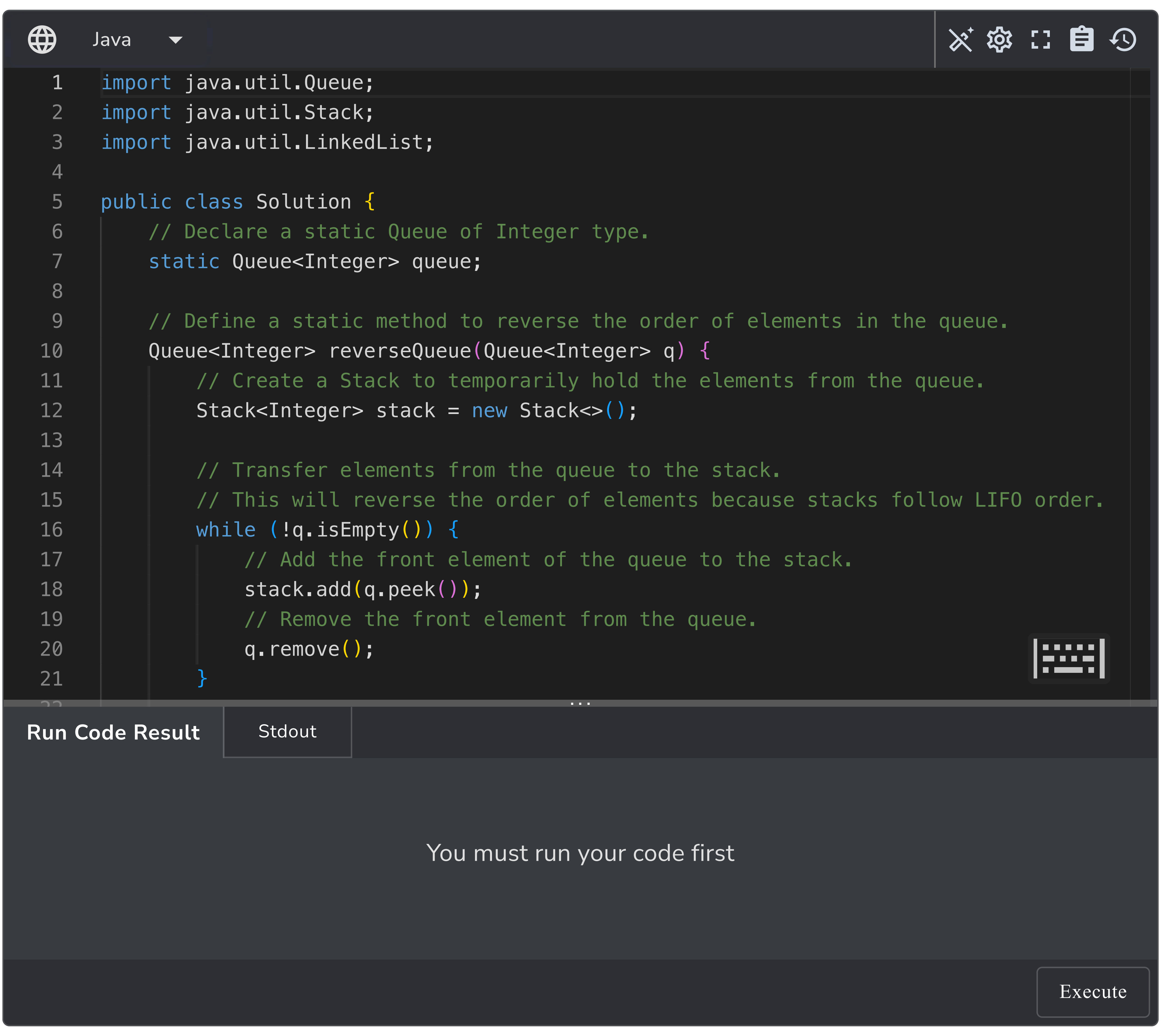
Here is the detailed step-by-step algorithm to reverse a queue:

1. Initialize an empty stack.
2. While the queue is not empty, dequeue an element from the queue and push it onto the stack.
3. Once all the elements have been transferred to the stack, pop elements from the stack and enqueue them back to the queue until the stack is empty.
4. The queue now contains the elements in reversed order.



Code

Here is how we can implement this algorithm:



```java
import java.util.Queue;
import java.util.Stack;
import java.util.LinkedList;

public class Solution {
    // Declare a static Queue of Integer type.
    static Queue<Integer> queue;

    // Define a static method to reverse the order of elements in the queue.
    Queue<Integer> reverseQueue(Queue<Integer> q) {
        // Create a Stack to temporarily hold the elements from the queue.
        Stack<Integer> stack = new Stack<>();

        // Transfer elements from the queue to the stack.
        // This will reverse the order of elements because stacks follow LIFO order.
        while (!q.isEmpty()) {
            // Add the front element of the queue to the stack.
            stack.add(q.peek());
            // Remove the front element from the queue.
            q.remove();
        }
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is O(n), where n is the number of elements in the queue. This is because each element is dequeued and enqueued exactly once.

The space complexity is also O(n), because all the elements are stored in the stack when reversing the queue.

← Previous
Problem 1: Reverse A Queue

Next →
Problem 2: Implement Stack Using Queues

☐ Mark as Completed